

**LAPORAN PRAKTIKUM**  
**POSTTEST (4)**  
**ALGORITMA PEMROGRAMAN LANJUT**

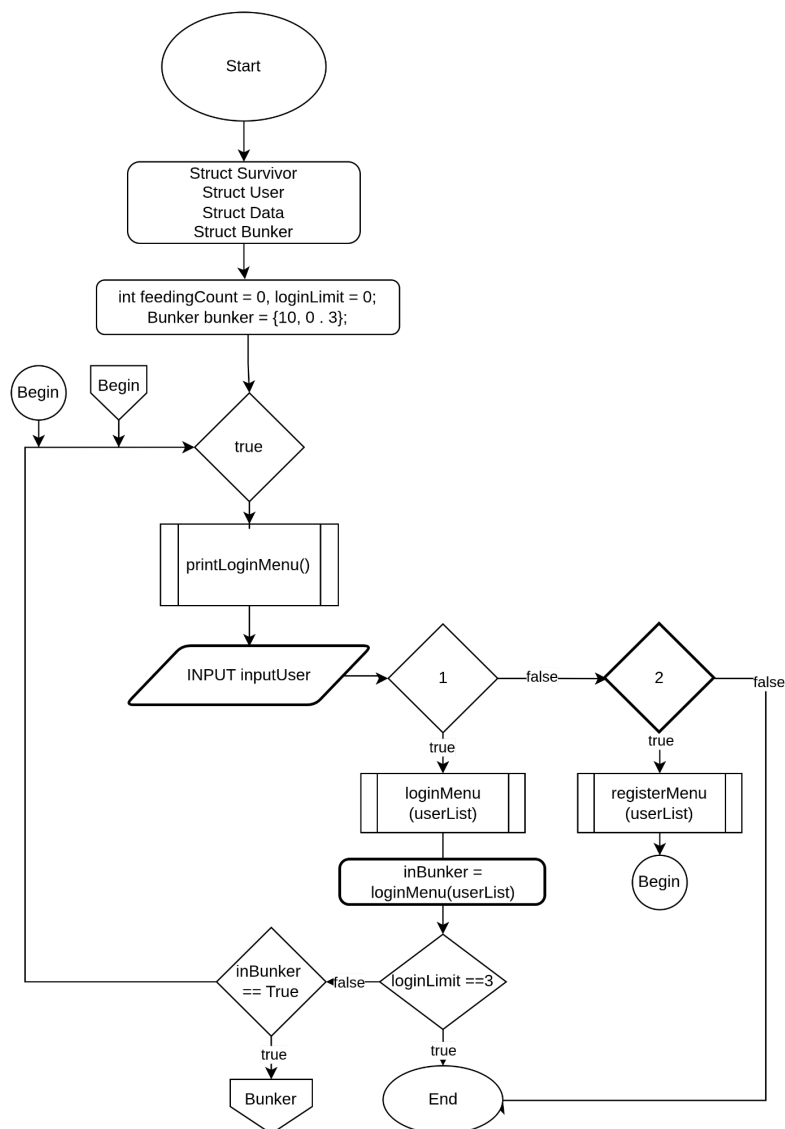


**Disusun oleh:**  
**Nama (2509106028)**  
**Kelas (A'25)**

**PROGRAM STUDI INFORMATIKA**  
**UNIVERSITAS MULAWARMAN**  
**SAMARINDA**  
**2026**

## 1. Flowchart

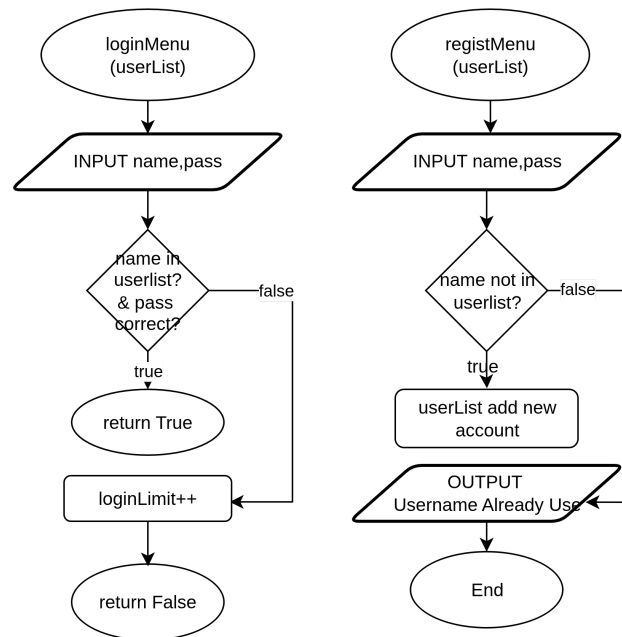
### 1.1. Menu Awal



**Gambar 1.1 Flowchart Menu Awal**

Pada menu awal ini program dimulai, user dapat memilih terlebih dahulu ingin login atau register terlebih dahulu, jika user gagal login selama tiga kali makan akan langsung dikeluarkan dari program, namun jika user berhasil login user akan dibawa masuk ke menu bunker.

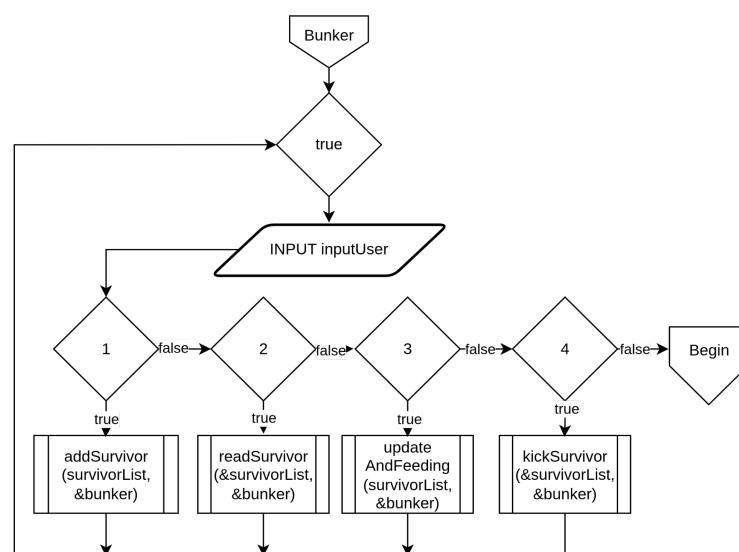
## 1.2. Fungsi dan Prosedur Login dan Register



**Gambar 1.2 Flowchart Login dan Register**

Pada fungsi login, pengguna diminta memasukkan username dan password. Jika data sesuai, pengguna akan diarahkan ke menu bunker. Begitu pula pada proses registrasi, pengguna diminta mengisi username dan password, jika data valid, registrasi dinyatakan berhasil.

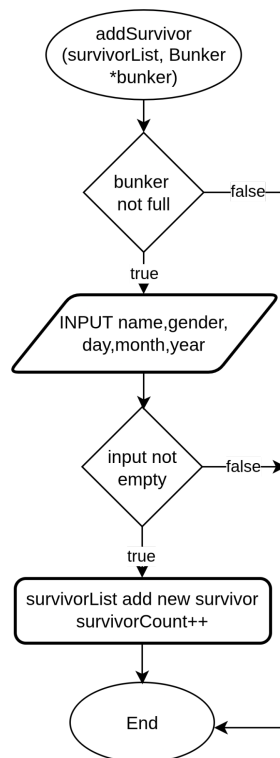
## 1.3. Menu Bunker



**Gambar 1.3 Flowchart Menu Bunker**

Pada menu bunker user dapat memilih untuk menambah, melihat, memperbarui dan menghapus survivor dari bunker, jika user tidak memilih dari keempatnya user akan di kembalikan ke menu awal.

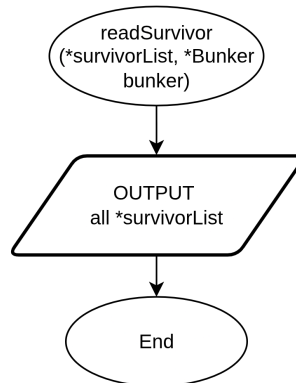
#### 1.4. Prosedur Tambah Survivor



**Gambar 1.4 Flowchart Prosedur Menambah Survivor**

Pada prosedur ini user dapat menambahkan survivor baru masuk ke dalam bunker beberapa data akan diminta seperti nama, gender, dan tanggal masuk survivor ke bunker namun jika inputan kosong maka penambahan survivor baru akan gagal.

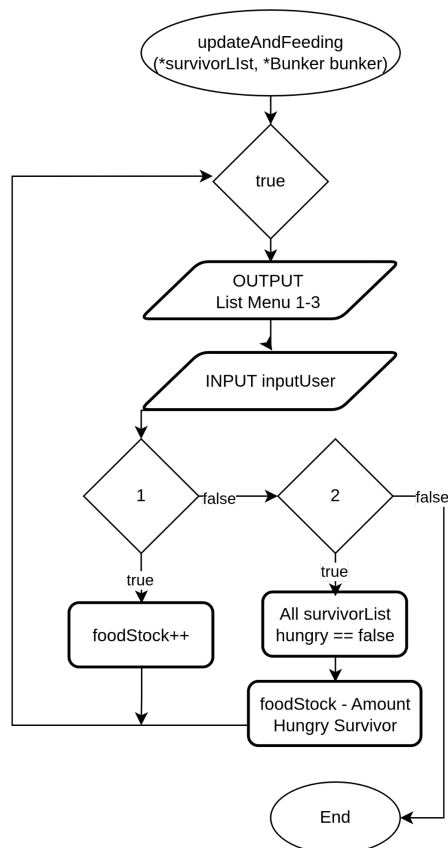
### 1.5. Prosedur Melihat List Survivor



**Gambar 1.5 Flowchart Prosedur Melihat List Survivor**

Pada prosedur ini akan menampilkan list survivor yang ada di dalam bunker.

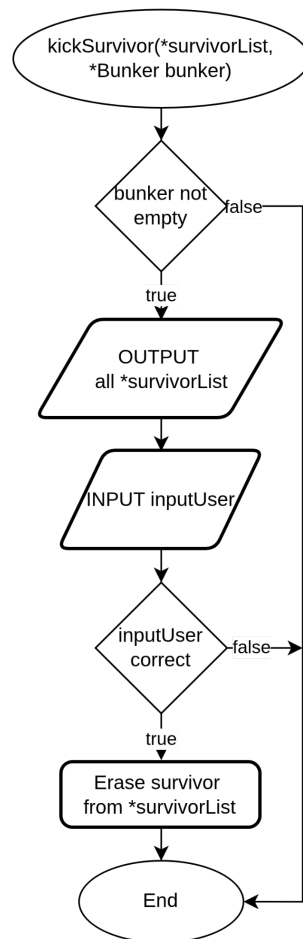
### 1.6. Prosedur Cari Makan dan Beri Makan



**Gambar 1.6 Flowchart Prosedur Cari Makan dan Beri Makan**

Pada prosedur ini user dapat mencari makan untuk menambah stok makanan di bunker, jika stok cukup user bisa memberi makan semua penghuni bunker agar status lapar bisa hilang.

### 1.7. Prosedur Tendang Survivor



**Gambar 1.7 Flowchart Tendang Survivor**

Pada prosedur tendang survivor seperti namanya user dapat melakukan penghapusan survivor dari list survivor dengan menendang keluar survivor tersebut yang dipilih.

## 2. Deskripsi Singkat Program

### 2.1. Sistem Keamanan & Akun

Program memiliki sistem login dengan dua level akses: **Admin** dan **User**.

- **Registrasi:** Pengguna baru dapat mendaftar jika slot akun maksimal masih tersedia.
- **Autentikasi:** Login dibatasi maksimal 3 kali percobaan kegagalan sebelum program otomatis tertutup.
- **Hak Akses:** Hanya Admin yang dapat menambah, memberi makan, atau mengeluarkan survivor. User biasa hanya diberikan izin untuk melihat daftar survivor.

## 2.2. Manajemen Data (Structs)

Program menggunakan struktur data (struct) yang saling berhubungan untuk mengorganisir informasi:

- **User:** Menyimpan username, password, dan tipe akun.
- **Date:** Menyimpan informasi tanggal (hari, bulan, tahun).
- **Survivor:** Menyimpan identitas survivor (nama, gender), status kesehatan (lapar/hidup), dan tanggal masuk ke bunker.
- **Bunker :** Menyimpan informasi stok makanan, kapasitas bunker, dan jumlah survivor yang ada.

## 2.3. Operasi Manajemen Bunker (CRUD)

Di dalam menu utama bunker, admin dapat melakukan pengelolaan logistik:

- **Add Survivor:** Menambah orang baru ke dalam bunker dengan batasan maksimal.
- **Read List:** Menampilkan daftar penghuni bunker beserta status rasa lapar mereka dan sisa stok makanan.
- **Update (Logistik):** Fitur untuk mencari makanan (menambah stok) dan memberi makan survivor yang lapar. Memberi makan akan mengubah status hungry dari true menjadi false.

- **Kick Survivor:** Menghapus data survivor dari daftar dengan sistem penggeseran indeks array.

## 2.4. Efisiensi Memori

Dalam mengurangi penggunaan memori dan mempercepat kecepatan program diberlakukan beberapa hal pada program yaitu penerapan penggunaan pointer sehingga terutama pada penggunaan fungsi dan prosedur tidak terus menerus membuat variabel baru yang memakan memori dan hanya dipakai sekali.

## 3. Source Code

### 3.1. Login

```
bool loginMenu(vector<User> &userList) {
    string name, pass;
    cout << "Input Username : ";
    cin.ignore(1000, '\n');
    getline(cin, name);
    cout << "Input Password : ";
    cin >> pass;
    for (auto &account : userList) {
        if (account.userName == name && account.userPass ==
pass) {
            cout << "Login Succes\n";
            typeUser = account.userType;
            return true;
        }
    }
    cout << "Login Failed\n";
    loginLimit++;
    return false;
}

void registerMenu(vector<User> &userList) {
    cout << "Input Name : ";
    cin.ignore(1000, '\n');
    getline(cin, name);
```



```

    cout << "Input Pass : ";
    cin >> pass;
    for (auto &account : userList) {
        if (name == account.userName) {
            cout << "Username Already Use\n";
            return;
        }
    }
    userList.push_back(User(name, pass, "user"));
    cout << "Register Succes\n";
}

```

### 3.2. Registrasi

```

void registerMenu(vector<User> &userList) {
    cout << "Input Name : ";
    cin.ignore(1000, '\n');
    getline(cin, name);
    cout << "Input Pass : ";
    cin >> pass;
    for (auto &account : userList) {
        if (name == account.userName) {
            cout << "Username Already Use\n";
            return;
        }
    }
    userList.push_back(User(name, pass, "user"));
    cout << "Register Succes\n";
}

```

### 3.3. Menambahkan Survivor

```

void addSurvivor(vector<Survivor> &survivorList, Bunker
*bunker, string n,
                string g, string d, string m, string y)
{
    if (bunker->survivorCount > bunker->limit - 1) {
        cout << "Bunker Full\n";
    }
}

```

```

    } else {
        if (n.empty() || g.empty() || d.empty() || m.empty()
|| y.empty()) {
            cout << "Input Cannot Empty\n";
        } else {
            survivorList.push_back(Survivor(n, g, d, m, y));
            bunker->survivorCount++;
            cout << "New Survivor Succes To Add\n";
        }
    }
}
}
}

```

### 3.4. Melihat List Survivor

```

void readSurvivor(vector<Survivor> *survivorList, Bunker
*bunker) {
    Table t;
    t.add_row({"No", "Name", "Gender", "Status"});

    t[0].format().font_style({FontStyle::bold}).font_align(FontAl
ign::center);
    if (bunker->survivorCount == 0) {
        t.add_row({"-", "Bunker Empty", "-", "-"});
    } else {
        for (int i = 0; i < bunker->survivorCount; i++) {
            t.add_row({to_string(i + 1),
(*survivorList)[i].survivorName,
(*survivorList)[i].survivorGender,
(*survivorList)[i].hungry ? "Hungry" : "Not
Hungry"});
        }
    }
    cout << t << endl;

    Table info;
    info.add_row({"Food Stock", bunker->foodStock == 0
? "Empty"
:
to_string(bunker->foodStock)});
    info[0].format().font_style({FontStyle::bold});
    if (bunker->foodStock == 0)
        info[0][1].format().font_color(Color::red);
    else

```

```

        info[0][1].format().font_color(Color::green);
    cout << info << endl;
}

```

### 3.5. Mencari Makanan atau Memberi Makan

```

void updateAndFeeding(vector<Survivor> &survivorList,
Bunker *bunker) {
    while (true) {
        feedingCount = 0;
        cout << "-----Update & Feeding
Menu-----\n";
        cout << "1. Search Food\n";
        cout << "2. Feeding Hungry Survivor\n";
        cout << "3. Exit\n";
        cout << "Input Id Menu : ";
        cin >> inputUser;
        if (inputUser == 1) {
            cout << "Search Food.....\n";
            bunker->foodStock += 3;
            cout << "Search Food Succes\n";
            cout << "Food Stock Now : " << bunker->foodStock
<< endl;
        } else if (inputUser == 2) {
            if (bunker->survivorCount == 0) {
                cout << "Bunker Empty\n";
                continue;
            }
            for (int i = 0; i < bunker->survivorCount; i++) {
                if (bunker->foodStock != 0 &&
survivorList[i].hungry == true) {
                    survivorList[i].hungry = false;
                    feedingCount++;
                    bunker->foodStock--;
                }
            }
            cout << "Survivor Get Feed : " << feedingCount <<
endl;
            cout << "Food Stock : " << bunker->foodStock <<
endl;
        } else if (inputUser == 3) {

```

```

        cout << endl;
        break;
    }
}
}

```

### 3.6. Menghapus Survivor Dari List

```

void kickSurvivor(vector<Survivor> *survivorList, Bunker
*bunker) {
    if (bunker->survivorCount == 0) {
        cout << "Bunker Empty\n";
        return;
    }
    Table t;
    t.add_row({"No", "Name"});

    t[0].format().font_style({FontStyle::bold}).font_align(F
ontAlign::center);
    for (int i = 0; i < bunker->survivorCount; i++) {
        t.add_row({to_string(i + 1),
(*survivorList)[i].survivorName});
    }
    cout << t << endl;
    cout << "Input [0] For Exit\n";
    cout << "Input Number Survivor to Kick From Bunker :
";
    cin >> inputUser;
    if (inputUser <= 0 || inputUser >
bunker->survivorCount) {
        return;
    }
    for (int i = inputUser - 1; i < bunker->survivorCount
- 1; i++) {
        (*survivorList)[i] = (*survivorList)[i + 1];
    }
    survivorList->pop_back();
    bunker->survivorCount--;
}

```

## 4. Hasil Output

### 4.1. Login

```
-----Login Menu-----  
1. Login  
2. Regist  
3. Exit  
Input Menu Id : 1  
Input Username : Akbar Rachim  
Input Password : 028  
Login Succes
```

Gambar 4.1 Hasil Output

### 4.2. Registrasi

```
-----Login Menu-----  
1. Login  
2. Regist  
3. Exit  
Input Menu Id : 2  
Input Name : Rudi  
Input Pass : 123
```

Gambar 4.2 Hasil Output Registrasi

### 4.3. Tambah Survivor

```
-----Bunker Menu-----  
1. Add Survivor  
2. Read List Survivor  
3. Update Food Or Survivor  
4. Kick Survivor  
5. Exit  
Input Menu Id : 1  
Name : Akbar Rachim  
Gender : Male  
Day : 10  
Month : 12  
Year : 2100
```

Gambar 4.3 Hasil Output Tambah Survivor

#### 4.4. Lihat List Survivor

```
+-----+
| No | Name | Gender | Status |
+-----+
| 1 | Budi | Male | Hungry |
+-----+
| 2 | Radi | Male | Hungry |
+-----+
| 3 | Gea | Female | Hungry |
+-----+
+-----+
[ ] Food Stock | 10 |
+-----+
```

Gambar 4.4 Hasil Output

#### 4.5. Mencari Makanan atau Memberi Makan

```
Search Food.....
Search Food Succes
Food Stock Now : 33
-----
1. Search Food
2. Feeding Hungry Survivor
3. Exit
Input Id Menu : 2
Survivor Get Feed : 1
Food Stock : 32
```

Gambar 4.5 Hasil Output

#### 4.6. Keluarkan Survivor dari Bunker

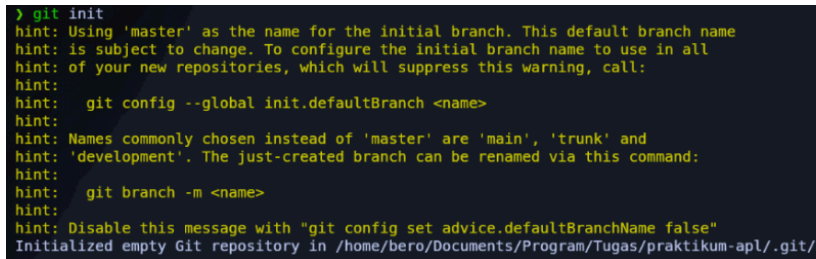
```
+-----+
| No | Name |
+-----+
| 1 | Budi |
+-----+
| 2 | Radi |
+-----+
| 3 | Gea |
+-----+
Input [0] For Exit
Input Number Survivor to Kick From Bunker : 1
```

```
-----E
1. Add Surv
2. Read List
3. Update Fo
4. Kick Surv
5. Exit
Input Menu 1
-----E
1. Akbar Rac
Input [0] Fo
Input Number
```

Gambar 4.6 Hasil Output

## 5. Langkah-langkah GIT

### 5.1. GIT Init



```
> git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/bero/Documents/Program/Tugas/praktikum-apl/.git/
```

Gambar 5.1 GIT Init

Membuat repository baru git untuk melacak setiap perubahan dalam folder sehingga setiap perubahan dapat dilihat.

### 5.2. GIT Remote

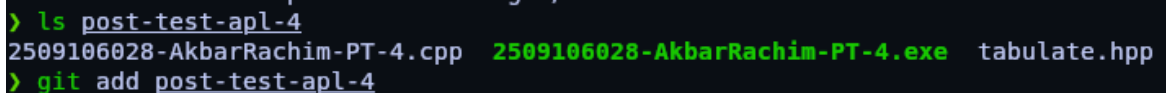


```
> git remote add origin https://github.com/cybercrym-crying/Praktikum-APL.git
```

Gambar 5.2 GIT Remote

Menghubungkan repository git lokal dengan cloud yaitu github.

### 5.3. GIT Add

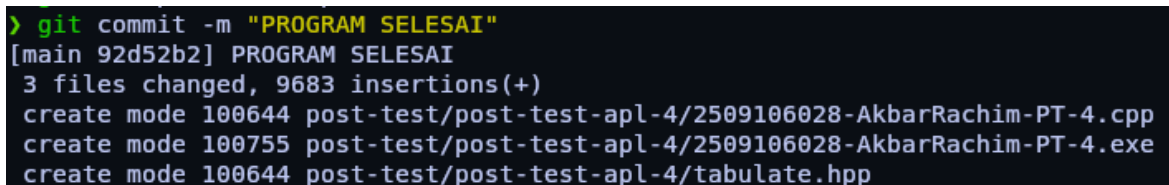


```
> ls post-test-apl-4
2509106028-AkbarRachim-PT-4.cpp 2509106028-AkbarRachim-PT-4.exe tabulate.hpp
> git add post-test-apl-4
```

Gambar 5.3 GIT Add

Melakukan git add untuk menambahkan file dan folder post-test-apl-3 ke stage untuk di commit.

### 5.4. GIT Commit



```
> git commit -m "PROGRAM SELESAI"
[main 92d52b2] PROGRAM SELESAI
3 files changed, 9683 insertions(+)
create mode 100644 post-test/post-test-apl-4/2509106028-AkbarRachim-PT-4.cpp
create mode 100755 post-test/post-test-apl-4/2509106028-AkbarRachim-PT-4.exe
create mode 100644 post-test/post-test-apl-4/tabulate.hpp
```

Gambar 5.4 Git Commit

Pada gambar 5.4 git commit mulai mencatat dan menyimpan rekaman perubahan file dan folder di ke dalam git.

## 5.5. GIT Push

```
> git push -u origin main
Username for 'https://github.com': cybercrym-crying
Password for 'https://cybercrym-crying@github.com':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 773.81 KiB | 4.58 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/cybercrym-crying/Praktikum-APL.git
 39dc75f..92d52b2  main -> main
```

**Gambar 5.5 GIT Push**

Pada langkah git terakhir gambar 5.5 git push akan Menyimpan perubahan git (local) menuju github (cloud) secara online.